

Trade-off between Small and Large Transformers for German NER Fine-Tuning

Luca Sander

January 10, 2026

Abstract

TODO

1 Introduction

TODO

2 Problem Statement and Research Questions

TODO

3 Background

TODO

4 Related Work

TODO

5 Dataset

The dataset we use in this study is the GermEval 2014 NER dataset [3], which builds upon the annotation published by Benikova [2], which after some adjustments became the dataset used for the GermEval 2014 shared task [1] and was downloaded from the Hugging Face datasets library. The dataset contains German text from Wikipedia and news articles, annotated with named entities in a nested format. In total the dataset contains over 30,000 sentences with approximately 590,000 tokens. The dataset is split into:

- Training set: 24,002 sentences
- Development set: 2,200 sentences

- Test set: 5,100 sentences

The dataset includes annotations for 4 main entity types: *Person*, *Organization*, *Location*, and *Other*. Each entity type also has subtypes, namely *part* and *deriv*, which leads to a total of 12 distinct entity labels. The dataset also uses nested annotations, meaning that entities can be contained within other entities. An example of that would be "*FC BAYERN MUNICH*", where "*MUNICH*" is firstly annotated as an *Organization*, but in a nested annotations it is also annotated as a *Location*. The distribution of the annotated entities in the training set is shown in Table 1.

Entity Type	All	Nested
Location	12,204	1,454
Location deriv	4,412	808
Location part	713	39
Person	10,517	488
Person deriv	95	20
Person part	275	29
Organization	7,182	281
Organization deriv	56	4
Organization part	1,077	9
Other	4,047	57
Other deriv	294	3
Other part	252	2
Total	41,124	3,194

Table 1: Distribution of entity types in the GermEval 2014 dataset (outer and nested annotations).

However, for the purpose of this study, we only focus on the outermost entities and ignore the nested annotations. Concretely, only the primary annotation layer (outer entities) is retained, while the nested entity layer is not used. This choice reduces the task to a standard flat named entity

recognition setting, where each token is assigned at most one entity label. This follows the more general application of transformer-based Named Entity Recognition, where other datasets often do not have the nested elements, such as the CoNLL-2003 dataset [11] used for example in English NER. [8].

6 Approach

6.1 Models

For this study, we selected three transformer-based models that vary in size and purpose. All models were downloaded from the Hugging Face model hub. The models chosen are:

- **bert-base-german-cased** [4]: The German BERT base model is a transformer model published in 2019, pretrained on roughly 12GB of German text data, sensitive to cased input, and evaluated on various tasks including CONLL03, GermEval14, and GNAD. The model has 12 layers, each with 12 attention heads, the tokens are embedded into a 768-dimensional space, with the feed-forward layers having a dimensionality of 3072. The vocabulary size of the model is 30,000 tokens, leading to a total of around 110 million parameters.
- **distilbert-base-german-cased** [6]: DistilBERT is a compressed version of BERT trained through knowledge distillation, where a smaller student model is trained to match the behavior of a larger teacher model. The DistilBERT model has the same hidden dimensionality, feed-forward size, and number of attention heads as the BERT base model, but reduces the number of transformer layers from 12 to 6, resulting in a significantly smaller and faster model, while retaining the same width. The model uses a vocabulary of 31,102 tokens and contains approximately 67 million parameters.
- **bert-base-multilingual-cased** [5]: The multilingual BERT base model is a transformer model released by Google, pretrained with a masked language modeling objective on the Wikipedias of 104 languages (including German), and is case sensitive. It follows the BERT-base architecture with 12 layers and 12 attention heads, a hidden size of 768 and a

feed-forward size of 3072. Due to its much larger vocabulary (119,547 tokens), the model has approximately 179 million parameters.

6.2 Training Strategies

We use two different training strategies for fine-tuning all models on the GermEval14 dataset. In all cases, we add a token classification head as a linear layer on top of the transformer to predict the entity labels for each token. We then experiment with the following strategies for training:

- **Full Fine-Tuning**: In this strategy, we train the entire model, including all transformer layers and the classification head, on the GermEval14 training set. This allows the model to adapt all its parameters to the specific NER task.
- **Frozen Encoder**: In this strategy, we freeze all the transformer layers and only train the classification head on top. This means that the pre-trained weights of the transformer remain unchanged, and only the final layer is updated during training. This approach is computationally more efficient and requires less memory, as only the parameters of the classification head need to be stored and updated.

We also experiment with an additional variant where we concatenate the last four hidden layers of the transformer as input to the classification head, instead of just using the final layer’s output. This approach needs a modification of the classification head to accommodate the increased input dimensionality, so its size is $4 * 768 = 3072$.

This choice is motivated by prior work showing that combining representations from multiple layers can improve performance on NER tasks. [10].

The specific hyperparameter configurations for each strategy were selected based on a small grid search on the development set, as described in Section 8.

6.3 Implementation Details

Label set and preprocessing The label inventory is derived from the GermEval14 dataset by collecting all unique BIO tags (`ner_tag`) across all dataset splits. A deterministic mapping between labels and numeric IDs (`label2id/id2label`) is created and passed to the model configuration to ensure consistent decoding and evaluation.

Tokenization and label alignment All models use their respective tokenizers via `AutoTokenizer`. Since the dataset provides word-level token sequences, tokenization is performed with `is_split_into_words=True`, applying truncation and fixed-length padding up to a maximum sequence length of 128. Label alignment is implemented using the tokenizer’s `word_ids()` mapping: special tokens and padding positions (e.g., `[CLS]`, `[SEP]`) are masked with the ignore index `-100`. For words that are split into multiple subword tokens, the label is assigned only to the first subword token, while all subsequent subwords are masked with `-100`. This follows the standard Hugging Face token-classification preprocessing and ensures that loss and evaluation are computed only on one token per original word. [7]

Model setup Models are instantiated using `AutoModelForTokenClassification`. Depending on the training strategy, either all model parameters are updated (full fine-tuning) or only the parameters of the token classification head are optimized while the transformer encoder is frozen.

Training Loop Training is implemented using the Hugging Face Trainer API, which provides a standardized training and evaluation loop for transformer models. Evaluation is performed at the end of each epoch on the development set. Final model performance is assessed on the test set; the evaluation metrics and protocol are described in detail in Section 7.

7 Evaluation

7.1 Metrics (GermEval14)

Benikova et al.[1] define four metrics for the shared task, but describe three main evaluation variants for nested NER. We summarize the three proposed metrics below.

Let P_1 and G_1 be the predicted/gold sets of outer (first-level) named entity chunks, and P_2 and G_2 the predicted/gold sets of inner (second-level) chunks. [1]

- **Metric 1: Strict, Combined Evaluation (official).** All markables across both levels are

evaluated jointly, distinguishing all 12 labels (including `deriv` and `part` subtypes). A predicted chunk matches a gold chunk iff (i) label matches, (ii) span matches, and (iii) the level (outer vs. inner) matches: $P = P_1 \cup P_2$, $G = G_1 \cup G_2$, $p == g \Leftrightarrow \text{label}(p) = \text{label}(g) \wedge \text{span}(p) = \text{span}(g) \wedge \text{level}(p) = \text{level}(g)$.

- **Metric 2: Loose, Combined Evaluation.** Same as Metric 1, but subtypes are collapsed (base label match is sufficient, e.g. `PER` matches `PERderiv`), while span and level must still match: $P = P_1 \cup P_2$, $G = G_1 \cup G_2$, $p == g \Leftrightarrow \text{baseLabel}(p) = \text{baseLabel}(g) \wedge \text{span}(p) = \text{span}(g) \wedge \text{level}(p) = \text{level}(g)$.
- **Metric 3: Strict, Separate Evaluation.** Two separate strict evaluations are reported: one for outer entities (P_1 vs. G_1) and one for inner entities (P_2 vs. G_2), each using the traditional strict match definition (label + exact span).

7.2 What we evaluate in this study

In our experiments we report:

- **Entity-level Precision, Recall, and F1-score** computed from predicted and gold chunks of the outer entities using strict matching (exact span and label). We report micro-averaged scores as the primary metric for model comparison, corresponding to Metric 3’s outer evaluation and focusing on the standard flat NER task. In addition, we compute per-class and macro-averaged scores for diagnostic analysis, which are reported in Appendix D.
- **Training Runtime** (wall-clock) for each experiment, to quantify efficiency trade-offs between model size and training strategy.
- **Inference Runtime** (GPU execution) on the test set, to assess practical deployment efficiency.

7.3 Evaluation protocol and outer-only choice

The GermEval dataset provides two annotation layers and the shared task evaluates both levels. However, in this study we only evaluate the outer

entities and ignore the nested layer, as this would add complexity that is not central to the research questions.

Concretely, we evaluate predicted chunks against G_1 (outer gold entities) and do not model nor score G_2 (inner entities). This choice is motivated by:

- **Task alignment and comparability:** most widely used NER benchmarks - including the CoNLL-2003 dataset [11] - assume flat annotation; focusing on outer entities reduces GermEval to this standard setting.
- **Modeling scope:** nested prediction introduces an additional structural requirement (two-level outputs) beyond plain sequence tagging; our experiments target the trade-off between model size and fine-tuning strategy under a standard token-classification setup.
- **Data characteristics:** only a minority of entities are nested (about 7.8% in the dataset statistics reported by the task paper), while inner entities show substantially lower performance in the shared task analysis, suggesting higher difficulty. [1]

8 Experimental Setup

8.1 Hardware and environment

All experiments were run on a single workstation equipped with an NVIDIA RTX 3080 GPU. Even though better hardware is available, the experiments still give insight into the relative efficiency of the models and training strategies.

8.2 Hyperparameter selection via small grid search

To ensure a fair and computationally feasible comparison across all twelve model configurations (three architectures $\times$ frozen vs. full fine-tuning $\times$ last layer vs. last-4-concatenation), we performed a small but targeted hyperparameter search on a single reference model: bert-base-german-cased. We conducted the search separately for the two training regimes:

- Full fine-tuning** (encoder + classifier head trainable), and
- Frozen encoder** (only the classifier head trainable).

We did not perform hyperparameter tuning for the classification using a concatenation of the last 4 layers, as this would remove comparability between the configurations.

To keep exploratory runs efficient, we trained on a reduced subset of 50% of the training split while always evaluating on the full development set. For each candidate configuration, we trained for the specified number of epochs and selected the setting with the highest development F1-score (outer entities, strict matching as described in Section 7). We additionally recorded wall-clock runtime to ensure the chosen settings were not only accurate but also practically feasible.

Search space. For full fine-tuning, we tested learning rates $\{2 \times 10^{-5}, 3 \times 10^{-5}, 5 \times 10^{-5}\}$ and epochs $\{3, 4, 5\}$. For the frozen encoder regime, we tested higher learning rates - $\{3 \times 10^{-4}, 5 \times 10^{-4}, 1 \times 10^{-3}, 2 \times 10^{-3}, 3 \times 10^{-3}\}$ - appropriate for training only a shallow head, and epochs in a slightly wider range - $\{5, 8, 10\}$ - to allow convergence under frozen representations.

Selected hyperparameters. We selected the following hyperparameters based on the grid search results:

- **Full fine-tuning:** learning rate 2×10^{-5} , 4 epochs.
- **Frozen encoder:** learning rate 2×10^{-3} , 10 epochs.

The full grid-search results are reported in Appendix A (Table 12).

These hyperparameters were then used unchanged for all models within the corresponding training regime to preserve comparability between architectures and to avoid confounding results with per-model tuning.

9 Results

In this section, we present the main results of our experiments. Unless stated otherwise, reported Precision, Recall, and F1-scores refer to micro-averaged entity-level results on outer entities only. We first report results for the full fine-tuning regime, followed by the frozen encoder regime. Finally, we present results for the last-4-layer concatenation variant. For readability, we refer to the

models *distilbert-base-german-cased*, *bert-base-german-cased*, and *bert-base-multilingual-cased* as *DistilBERT (DE)*, *BERT (DE)*, and *BERT (Multi)*, respectively, throughout this section.

9.1 Full Fine-tuning

We begin with the results obtained under the full fine-tuning regime.

9.1.1 Precision/Recall/F1-Score

Model	Prec.	Rec.	F1
DistilBERT (DE)	0.8427	0.8538	0.8482
BERT (DE)	0.8595	0.8626	0.8610
BERT (Multi)	0.8582	0.8692	0.8637

Table 2: Full fine-tuning results on the GermEval14 test set (outer entities, strict matching).

Table 2 reports Precision, Recall, and F1-scores for all three models under the full fine-tuning regime. BERT (DE) achieves the highest F1-score (0.8337), followed closely by BERT (Multi) (0.8308), while DistilBERT (DE) attains an F1-score of 0.8224. Overall, the distilled model performs approximately 1.1 F1 points below the base-sized models in this setting. The multilingual model slightly underperforms the German-specific BERT, although the difference is marginal.

9.1.2 Training Runtime

Model	Runtime (sec)
DistilBERT (DE)	409
BERT (DE)	791
BERT (Multi)	836

Table 3: Full fine-tuning training runtime on the GermEval14 dataset.

Training runtimes under full fine-tuning are summarized in Table 3. DistilBERT (DE) exhibits the shortest training time (413 seconds), followed by BERT (DE) (777 seconds) and BERT (Multi) (842 seconds). Overall, the distilled model requires roughly half the training time of the base-sized models, while the multilingual model incurs a modest additional cost compared to the German-specific BERT.

9.1.3 Inference Time

Model	Inference Time (s)
DistilBERT (DE)	5.39
BERT (DE)	12.26
BERT (Multi)	11.42

Table 4: Full fine-tuning inference time on the GermEval14 test set.

Inference runtimes for the full fine-tuning models are presented in Table 4. DistilBERT (DE) achieves the fastest inference time (3.70 seconds), while BERT (Multi) (6.78 seconds) outperforms BERT (DE) (8.07 seconds) in this regard.

9.2 Frozen Encoder

We now turn to the frozen encoder regime.

9.2.1 Precision/Recall/F1-Score

Model	Prec.	Rec.	F1
DistilBERT (DE)	0.7518	0.7739	0.7627
BERT (DE)	0.7485	0.7593	0.7539
BERT (Multi)	0.7497	0.7441	0.7469

Table 5: Frozen encoder results on the GermEval14 test set (outer entities, strict matching).

Training results for the frozen encoder regime are summarized in Table 5. Here, DistilBERT (DE) achieves the highest F1-score (0.6893), followed by BERT (DE) (0.6677) and BERT (Multi) (0.6607). Overall, the performance drop compared to full fine-tuning is substantial across all models, with F1-scores decreasing by approximately 13-17 points. BERT (DE) maintains a slight advantage over BERT (Multi).

9.2.2 Training Runtime

Model	Runtime (sec)
DistilBERT (DE)	366
BERT (DE)	691
BERT (Multi)	727

Table 6: Frozen encoder training runtime on the GermEval14 dataset.

Table 6 presents training runtimes for the frozen encoder models. Compared to full fine-tuning, training times are reduced across all models, with DistilBERT (DE) requiring 120 seconds less - a 29% reduction -, while BERT (DE) and BERT (Multi) see reductions of 172 seconds (22%) and 248 seconds (29%), respectively.

9.2.3 Inference Time

Model	Inference Time (s)
DistilBERT (DE)	5.82
BERT (DE)	11.54
BERT (Multi)	11.64

Table 7: Frozen encoder inference time on the GermEval14 test set.

Inference runtimes for the frozen encoder models are shown in Table 7. Compared to full fine-tuning, inference times are slightly increased for DistilBERT (DE) (4.15 seconds vs. 3.70 seconds), while BERT (DE) and BERT (Multi) see minor decreases (6.59 seconds vs. 8.07 seconds and 6.70 seconds vs. 6.78 seconds, respectively). Overall, inference times remain comparable across training regimes.

9.3 Last-4-Layer Concatenation

Below we present results for the variant where the last four hidden layers of the transformer are concatenated as input to the classification head, instead of using only the final layer’s output. However, we only report the Precision/Recall/F1-scores here, as training and inference runtimes are similar to the standard setups already reported, but can be found in Appendix B and C.

9.3.1 Full Fine-tuning

Model	Prec.	Rec.	F1
DistilBERT (DE)	0.8424	0.8538	0.8481
BERT (DE)	0.8556	0.8606	0.8581
BERT (Multi)	0.8574	0.8674	0.8624

Table 8: Full fine-tuning with last-4-layer concatenation results on the GermEval14 test set (outer entities, strict matching).

Table 8 summarizes the results for the last-4-layer concatenation variant under full fine-tuning. BERT (DE) achieves the highest F1-score (0.8355), followed by BERT (Multi) (0.8323) and DistilBERT (DE) (0.8197). Compared to the standard full fine-tuning results, BERT (DE) and BERT (Multi) see slight improvements in F1-score, while DistilBERT (DE) experiences a minor decrease, but overall the differences are marginal.

9.3.2 Frozen Encoder

Model	Prec.	Rec.	F1
DistilBERT (DE)	0.7716	0.7988	0.7850
BERT (DE)	0.7574	0.7859	0.7714
BERT (Multi)	0.7852	0.8051	0.7950

Table 9: Frozen encoder with last-4-layer concatenation results on the GermEval14 test set (outer entities, strict matching).

The results for the last-4-layer concatenation variant under the frozen encoder regime are presented in Table 9. The highest F1-score is achieved by DistilBERT (DE) (0.7326), followed by BERT (Multi) (0.7318) and BERT (DE) (0.7122). Compared to the standard frozen encoder results, all models see notable improvements in F1-score, with increases of approximately 5-7 points. However, compared to full fine-tuning, performance remains substantially lower.

Note on variability. Small deviations from expected trends may in part be attributed to stochastic effects inherent in neural network training, including random initialization and optimization noise. As all experiments were conducted using a single random seed, minor fluctuations in relative performance should be interpreted with caution.

10 Discussion

This section discusses the observed performance and efficiency differences across training regimes, model sizes, and architectural variants, with a particular focus on how these differences relate to properties of the Transformer architecture and training dynamics.

Effect of training regime. Across all evaluated models, full fine-tuning substantially outperforms the frozen encoder regime, with F1-score reductions of approximately 13–17 points when freezing the encoder. This indicates that task-specific adaptation of contextual representations is critical for high-quality German NER. When the encoder is frozen, only the classification head can adapt to the task, limiting the model’s ability to refine token-level representations and boundary decisions. As a result, even large pretrained encoders are unable to fully compensate for the lack of encoder-level adaptation.

Training runtime and parameter updates.

Freezing the encoder leads to a consistent reduction in training runtime across all models, which can be attributed to the reduced number of trainable parameters and the elimination of backpropagation through the encoder layers. While the forward pass remains unchanged, the backward pass and optimizer updates are restricted to the classification head, resulting in lower computational cost per training step. The relative reduction is particularly pronounced for DistilBERT, where the encoder already has fewer layers and the remaining training cost is dominated by the classifier, leading to a larger proportional runtime decrease. Between the two base-sized models, BERT (Multi) exhibits slightly longer training times than BERT (DE), while inference runtimes remain comparable, reflecting their identical architectural depth and similar forward-pass computational complexity, with differences in vocabulary size having negligible impact on runtime compared to sequence length and hidden dimensionality.

Inference time behavior. Inference times are largely determined by the forward pass through the encoder and are therefore mostly independent of the training regime. This explains why inference runtimes remain comparable between full fine-tuning and frozen encoder setups. Furthermore, the similar inference times observed for BERT (DE) and BERT (Multi) can be explained by their identical architectural depth and hidden dimensionality, resulting in comparable computational complexity per input sequence. In contrast, DistilBERT achieves substantially faster inference due to its reduced number of layers, despite sharing the same hidden size.

Model size, efficiency, and practical trade-offs.

Although DistilBERT exhibits a modest performance gap compared to base-sized models under full fine-tuning, this gap remains relatively small when contrasted with the significant gains in training and inference efficiency. Notably, a fully fine-tuned DistilBERT clearly outperforms a frozen encoder setup using a larger BERT model, highlighting that adapting a smaller model end-to-end is more effective than relying on a larger but non-adaptive encoder. This suggests that, under constrained computational budgets, fine-tuning a compact Transformer may be a more effective strategy than freezing a larger architecture.

Impact of last-4-layer concatenation.

Concatenating the last four hidden layers consistently improves performance under the frozen encoder regime and yields small gains for base-sized models under full fine-tuning. This indicates that exposing the classifier to representations from multiple abstraction levels can partially mitigate the loss of encoder adaptation by providing richer contextual signals. However, the observed improvements remain limited and do not close the performance gap to full fine-tuning, suggesting that architectural modifications alone cannot fully replace encoder-level learning.

Summary. Overall, the results demonstrate that performance in German NER is primarily driven by encoder adaptability rather than model size alone. While larger models provide higher absolute performance when fully fine-tuned, smaller models such as DistilBERT offer a strong efficiency–performance trade-off and benefit disproportionately from end-to-end fine-tuning. Lightweight architectural extensions, such as last-4-layer concatenation, can improve robustness in restricted training settings but remain secondary to full encoder adaptation.

11 Error Analysis

To better understand where performance differences originate, we analyze per-class behavior using classification reports. Since GermEval14 contains several closely related labels (e.g., base entities, derived forms, and part entities), we aggregate per-class F1-scores into three semantically motivated groups: base entities

(LOC/ORG/PER/OTH), derived entities (*deriv), and part entities (*part). Tables 10 and 11 report support-weighted grouped F1-scores for the full fine-tuning and frozen encoder regimes, respectively. Full per-class reports for all configurations are provided in Appendix D.

Model	Base	Deriv	Part
DistilBERT (DE)	0.857	0.858	0.641
BERT (DE)	0.870	0.870	0.686
BERT (Multi)	0.875	0.858	0.699

Table 10: Grouped, support-weighted F1-scores under full fine-tuning. Entity groups correspond to base, derived, and part entities.

Model	Base	Deriv	Part
DistilBERT (DE)	0.765	0.813	0.562
BERT (DE)	0.753	0.810	0.584
BERT (Multi)	0.752	0.770	0.516

Table 11: Grouped, support-weighted F1-scores under the frozen encoder regime.

Base vs. part entities. Across all models, base entities achieve substantially higher F1-scores than part entities. Under full fine-tuning, base entities are strong for all models (around 0.84-0.86), whereas part entities remain considerably harder (around 0.66-0.69). Under frozen encoders, this gap widens: base entity performance drops to roughly 0.67-0.70, while part entities degrade most severely (roughly 0.37-0.44). This suggests that resolving part labels relies more heavily on task-specific adaptation and fine-grained contextual cues (e.g., boundary decisions and compositional patterns) than recognizing core entity types.

Derived entities and rarity effects. Derived entities perform closer to base entities than part entities in both regimes, and under encoder freezing (Table 11) they have an even higher F1-score compared to the base entities. However, this aggregate hides strong label-frequency effects: extremely rare categories such as ORGderiv are consistently near-zero, indicating that data scarcity can dominate performance regardless of model size. In contrast, more frequent derived classes (e.g., LOCderiv) remain comparatively well handled.

Impact of training regime on error patterns.

Comparing Tables 10 and 11 shows that freezing the encoder disproportionately harms the most fine-grained distinctions, particularly the *part classes. This aligns with the overall regime-level results: restricting learning to the classification head limits the model’s ability to adapt token representations needed for subtle distinctions, whereas full fine-tuning improves both boundary sensitivity and label separability.

Model differences. Under full fine-tuning, base-sized models slightly outperform DistilBERT for the more challenging groups, especially part entities (Table 10). Under frozen encoders, DistilBERT exhibits stronger grouped performance on base and derived entities than the base-sized models, but part entities remain weak across all models. Overall, the grouped analysis supports the conclusion that encoder adaptability is the key driver of improvements for the hard classes.

12 Limitations

TODO

13 Conclusion and Future Work

TODO

References

- [1] Darina Benikova, Chris Biemann, Max Kisselew, and Sebastian Pado. GermEval 2014 Named Entity Recognition Shared Task: Companion Paper. .
- [2] Darina Benikova, Chris Biemann, and Marc Reznicek. NoSta-D Named Entity Annotation for German: Guidelines and Dataset. .
- [3] Darina Benikova et al. Germeval 2014 named entity recognition dataset. https://huggingface.co/datasets/lrschoen/germeval14_ner, 2014. Hugging Face dataset repository, accessed 2025.
- [4] Google Research. bert-base-german-cased. <https://huggingface.co/google-bert/>

- bert-base-german-cased, 2019. Pre-trained BERT base model for German (cased), accessed 2025.
- [5] Google Research. bert-base-multilingual-cased. <https://huggingface.co/google-bert/bert-base-multilingual-cased>, 2019. Pre-trained multilingual BERT model (cased), accessed 2025.
- [6] Hugging Face. distilbert-base-german-cased. <https://huggingface.co/distilbert/distilbert-base-german-cased>, 2020. Pre-trained DistilBERT model for German (cased), accessed 2025.
- [7] Hugging Face. Token classification. https://huggingface.co/docs/transformers/tasks/token_classification, n.d. Accessed: 2025.
- [8] Navjeet Kaur, Ashish Saha, Makul Swami, Muskan Singh, and Ravi Dalal. Bert-Ner: A Transformer-Based Approach For Named Entity Recognition. In *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, pages 1–7, June 2024. doi: 10.1109/ICCCNT61001.2024.10724703. URL <https://ieeexplore.ieee.org/document/10724703/>. ISSN: 2473-7674.
- [9] Raphael Scheible, Johann Frei, Fabian Thomczyk, Henry He, Patric Tippmann, Jochen Knaus, Victor Jaravine, Frank Kramer, and Martin Boeker. GottBERT: a pure German Language Model. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 21237–21250, Miami, Florida, USA, November 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.1183. URL <https://aclanthology.org/2024.emnlp-main.1183/>.
- [10] Fabio Tamburini. Solving Downstream Tasks with Transformers, 2024. URL <https://virtuale.unibo.it/course/view.php?id=69185>. Published: Lecture slides.
- [11] Erik F. Tjong Kim Sang and Fien De Meulder. Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition. In *Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003*, pages 142–147, 2003. URL <https://aclanthology.org/W03-0419/>.
- [12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is All you Need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html.

Regime	LR	Epochs	Val Prec.	Val Rec.	Val F1
Full fine-tuning					
Full FT	2×10^{-5}	3	0.8294	0.8691	0.8488
Full FT	2×10^{-5}	4	0.8294	0.8691	0.8589
Full FT	2×10^{-5}	5	0.8406	0.8680	0.8541
Full FT	3×10^{-5}	3	0.8222	0.8699	0.8454
Full FT	3×10^{-5}	4	0.8500	0.8628	0.8563
Full FT	3×10^{-5}	5	0.8400	0.8684	0.8540
Full FT	5×10^{-5}	3	0.8221	0.8609	0.8411
Full FT	5×10^{-5}	4	0.8407	0.8568	0.8487
Full FT	5×10^{-5}	5	0.8379	0.8620	0.8498
Frozen encoder					
Frozen	3×10^{-4}	5	0.7047	0.7307	0.7175
Frozen	3×10^{-4}	8	0.7111	0.7364	0.7235
Frozen	3×10^{-4}	10	0.7117	0.7386	0.7249
Frozen	5×10^{-4}	5	0.7067	0.7453	0.7255
Frozen	5×10^{-4}	8	0.7195	0.7483	0.7336
Frozen	5×10^{-4}	10	0.7175	0.7494	0.7331
Frozen	1×10^{-3}	5	0.7146	0.7584	0.7358
Frozen	1×10^{-3}	8	0.7213	0.7558	0.7381
Frozen	1×10^{-3}	10	0.7201	0.7580	0.7386
Frozen	2×10^{-3}	5	0.7092	0.7588	0.7332
Frozen	2×10^{-3}	8	0.7179	0.7528	0.7349
Frozen	2×10^{-3}	10	0.7222	0.7614	0.7413
Frozen	3×10^{-3}	5	0.7010	0.7532	0.7262
Frozen	3×10^{-3}	8	0.7164	0.7491	0.7324
Frozen	3×10^{-3}	10	0.7196	0.7603	0.7394

Table 12: Grid-search results for the bert-base-german-cased model. Validation precision, recall, and F1 are reported for outer entities using strict span matching. Best configurations per training regime are highlighted in bold.

Appendix

A Hyperparameter Search Results

B Training Time for Last-4-Layer Concatenation

Model	Runtime (sec)
DistilBERT (DE)	440
BERT (DE)	794
BERT (Multi)	787

Table 13: Training runtime for the last-4-layer concatenation variant under full fine-tuning on the GermEval14 dataset.

Model	Runtime (sec)
DistilBERT (DE)	371
BERT (DE)	717
BERT (Multi)	742

Table 14: Training runtime for the last-4-layer concatenation variant under the frozen encoder regime on the GermEval14 dataset.

C Inference Time for Last-4-Layer Concatenation

Model	Runtime (sec)
DistilBERT (DE)	5.66
BERT (DE)	10.49
BERT (Multi)	9.99

Table 15: Inference runtime for the last-4-layer concatenation variant under full fine-tuning on the GermEval14 test set.

Model	Runtime (sec)
DistilBERT (DE)	6.87
BERT (DE)	11.78
BERT (Multi)	11.86

Table 16: Inference runtime for the last-4-layer concatenation variant under the frozen encoder regime on the GermEval14 test set.

D Per-Class Results

Class	Supp.	Standard Full FT			Standard Frozen			Last-4 Full FT			Last-4 Frozen		
		P	R	F1	P	R	F1	P	R	F1	P	R	F1
LOC	1706	0.8856	0.9168	0.9009	0.8019	0.8710	0.8351	0.8869	0.9144	0.9004	0.8378	0.8869	0.8616
LOCderiv	561	0.8655	0.9519	0.9066	0.8270	0.8859	0.8554	0.8643	0.9537	0.9068	0.8264	0.9162	0.8690
LOCpart	109	0.7765	0.6055	0.6804	0.7361	0.4862	0.5856	0.7816	0.6239	0.6939	0.7765	0.6055	0.6804
ORG	1150	0.7954	0.8078	0.8016	0.6303	0.6522	0.6410	0.7884	0.8035	0.7959	0.6631	0.7017	0.6819
ORGderiv	8	0.0000	0.0000	0.0000	0.5000	0.1250	0.2000	0.0000	0.0000	0.0000	0.2500	0.1250	0.1667
ORGpart	172	0.7322	0.7791	0.7549	0.7754	0.6221	0.6903	0.7083	0.7907	0.7473	0.7467	0.6512	0.6957
OTH	697	0.6765	0.6571	0.6667	0.4675	0.4749	0.4712	0.6662	0.6643	0.6652	0.4877	0.5108	0.4989
OTHderiv	39	0.5455	0.6154	0.5783	0.5806	0.4615	0.5143	0.5750	0.5897	0.5823	0.5714	0.5128	0.5405
OTHpart	42	0.5455	0.2857	0.3750	0.5000	0.2143	0.3000	0.7692	0.2381	0.3636	0.4500	0.2143	0.2903
PER	1639	0.9239	0.9402	0.9320	0.8817	0.9274	0.9040	0.9298	0.9372	0.9335	0.8981	0.9304	0.9140
PERderiv	11	0.0000	0.0000	0.0000	0.3333	0.0909	0.1429	1.0000	0.0909	0.1667	0.2857	0.1818	0.2222
PERpart	44	0.4333	0.2955	0.3514	0.4211	0.1818	0.2540	0.5938	0.4318	0.5000	0.4545	0.2273	0.3030
Micro avg	6178	0.8427	0.8538	0.8482	0.7518	0.7739	0.7627	0.8424	0.8538	0.8481	0.7716	0.7988	0.7850
Macro avg	6178	0.5983	0.5712	0.5790	0.6212	0.4994	0.5328	0.7136	0.5865	0.6046	0.6040	0.5387	0.5604
Weighted avg	6178	0.8369	0.8538	0.8445	0.7464	0.7739	0.7577	0.8403	0.8538	0.8449	0.7683	0.7988	0.7821

Table 17: Per-class precision, recall, and F1-scores for DistilBERT (DE) across all training regimes and head variants. Aggregate scores (micro, macro, weighted) are computed over all entity classes.

Class	Supp.	Standard Full FT			Standard Frozen			Last-4 Full FT			Last-4 Frozen		
		P	R	F1	P	R	F1	P	R	F1	P	R	F1
LOC	1706	0.9064	0.9197	0.9130	0.8087	0.8623	0.8346	0.9053	0.9080	0.9066	0.8274	0.8822	0.8539
LOCderiv	561	0.8842	0.9394	0.9110	0.8248	0.8895	0.8559	0.8870	0.9376	0.9116	0.8451	0.8948	0.8693
LOCpart	109	0.7692	0.6422	0.7000	0.7436	0.5321	0.6203	0.7957	0.6789	0.7327	0.6979	0.6147	0.6537
ORG	1150	0.8033	0.8165	0.8098	0.6218	0.6348	0.6282	0.8053	0.8200	0.8126	0.6200	0.6739	0.6458
ORGderiv	8	1.0000	0.1250	0.2222	0.3333	0.1250	0.1818	0.6667	0.2500	0.3636	0.1667	0.1250	0.1429
ORGpart	172	0.7684	0.7907	0.7794	0.7681	0.6163	0.6839	0.7886	0.8023	0.7954	0.7566	0.6686	0.7099
OTH	697	0.7205	0.6915	0.7057	0.4394	0.4161	0.4274	0.6950	0.6801	0.6875	0.4765	0.4792	0.4778
OTHderiv	39	0.6176	0.5385	0.5753	0.4545	0.3846	0.4167	0.6486	0.6154	0.6316	0.5143	0.4615	0.4865
OTHpart	42	0.5312	0.4048	0.4595	0.3750	0.1429	0.2069	0.5484	0.4048	0.4658	0.4211	0.1905	0.2623
PER	1639	0.9340	0.9414	0.9377	0.8773	0.9121	0.8944	0.9244	0.9402	0.9322	0.8910	0.9225	0.9065
PERderiv	11	0.3750	0.2727	0.3158	0.3750	0.2727	0.3158	0.5000	0.4545	0.4762	0.2857	0.1818	0.2222
PERpart	44	0.5250	0.4773	0.5000	0.5862	0.3864	0.4658	0.5854	0.5455	0.5647	0.4706	0.3636	0.4103
Micro avg	6178	0.8595	0.8626	0.8610	0.7485	0.7593	0.7539	0.8556	0.8606	0.8581	0.7574	0.7859	0.7714
Macro avg	6178	0.7362	0.6300	0.6525	0.6007	0.5146	0.5443	0.7292	0.6698	0.6900	0.5811	0.5382	0.5534
Weighted avg	6178	0.8574	0.8626	0.8592	0.7415	0.7593	0.7488	0.8538	0.8606	0.8568	0.7543	0.7859	0.7690

Table 18: Per-class precision, recall, and F1-scores for BERT (DE) across all training regimes and head variants. Aggregate scores (micro, macro, weighted) are computed over all entity classes.

Class	Supp.	Standard Full FT			Standard Frozen			Last-4 Full FT			Last-4 Frozen		
		P	R	F1	P	R	F1	P	R	F1	P	R	F1
LOC	1706	0.9096	0.9138	0.9117	0.8072	0.8464	0.8263	0.9063	0.9127	0.9095	0.8551	0.8822	0.8684
LOCderiv	561	0.8565	0.9465	0.8992	0.7889	0.8396	0.8135	0.8649	0.9358	0.8990	0.8136	0.8948	0.8523
LOCpart	109	0.7708	0.6789	0.7220	0.8033	0.4495	0.5765	0.7184	0.6789	0.6981	0.7093	0.5596	0.6256
ORG	1150	0.7926	0.8209	0.8065	0.6266	0.6217	0.6242	0.8000	0.8209	0.8103	0.6984	0.7330	0.7153
ORGderiv	8	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	1.0000	0.1250	0.2222	0.3333	0.1250	0.1818
ORGpart	172	0.7419	0.8023	0.7709	0.7823	0.5640	0.6554	0.7337	0.7849	0.7584	0.6786	0.6628	0.6706
OTH	697	0.7431	0.7303	0.7366	0.4481	0.4275	0.4376	0.7541	0.7346	0.7442	0.5386	0.5610	0.5495
OTHderiv	39	0.5854	0.6154	0.6000	0.4390	0.4615	0.4500	0.5250	0.5385	0.5316	0.4667	0.5385	0.5000
OTHpart	42	0.5667	0.4048	0.4722	0.5000	0.0476	0.0870	0.5862	0.4048	0.4789	0.4737	0.2143	0.2951
PER	1639	0.9393	0.9439	0.9416	0.8845	0.9115	0.8978	0.9330	0.9433	0.9381	0.9077	0.9243	0.9160
PERderiv	11	0.5000	0.1818	0.2667	0.4000	0.1818	0.2500	0.3000	0.2727	0.2857	0.1667	0.1818	0.1739
PERpart	44	0.5814	0.5682	0.5747	0.4375	0.1591	0.2333	0.5714	0.5455	0.5581	0.4348	0.2273	0.2985
Micro avg	6178	0.8582	0.8692	0.8637	0.7497	0.7441	0.7469	0.8574	0.8674	0.8624	0.7852	0.8051	0.7950
Macro avg	6178	0.6656	0.6339	0.6418	0.5765	0.4592	0.4876	0.7244	0.6414	0.6528	0.5897	0.5421	0.5539
Weighted avg	6178	0.8563	0.8692	0.8622	0.7423	0.7441	0.7398	0.8566	0.8674	0.8613	0.7830	0.8051	0.7929

Table 19: Per-class precision, recall, and F1-scores for BERT (Multi) across all training regimes and head variants. Aggregate scores (micro, macro, weighted) are computed over all entity classes.